

Trabalho Prático 2
Comunicação e Sincronização de Processos e Threads

Grupo até 6 pessoas.

Observações:

Todos os programas devem imprimir o nome completo de todos os componentes do grupo. Os arquivos fontes dos programas devem conter um cabeçalho completo com entrada. Cópias grosseiras de programas serão desconsideradas.

Parte I Thread em JAVA

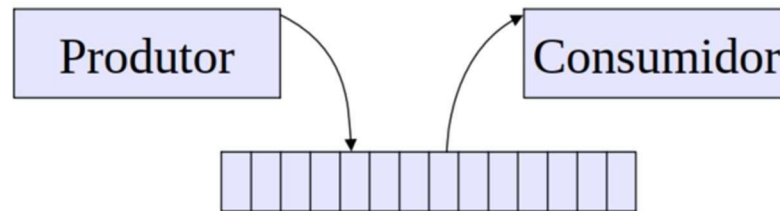
- 1) Edite, compile e execute os programas a seguir e depois explique (justificando) o que cada linha faz, com enfoque com o que será impresso na tela.

a) Problema do produtor e consumidor

O problema do produtor e consumidor, também referido como *Bounded Buffer Problem* (BBP), consiste em um desafio de sincronização de duas ou mais threads (tarefas) concorrentes que têm acesso a um mesmo recurso do programa que está sendo executado. O fato de o recurso estar sendo compartilhado, implica na

possibilidade de conflitos entre as threads que disputam pelo controle da CPU para acessar esse mesmo recurso.

No caso do BBP, o recurso disputado é um buffer de memória (um vetor), de tamanho limitado/fixo, daí o motivo do nome. O produtor é o responsável por enfileirar novos jobs ou dados nesse buffer, e o consumidor assume a responsabilidade de extrair esses dados do buffer.



Como ponto de partida, estude, compile e execute J\va d

Produtores e Consumidores

Classe “Buffer.JAVA”

```
1 package ProdutorConsumidor;
2
3 public class Buffer {
4     private int valor;
5     private boolean cheio = false;
6
7     public synchronized int get() {
8         while (!cheio) {
9             try {
10                 wait();
11             } catch (InterruptedException e) {
12             }
13         }
14         cheio = false;
15         notifyAll();
16         System.out.println("get " + valor);
17         return valor;
18     }
19
20     public synchronized void put(int valor) {
21         while (cheio) {
22             try {
23                 wait();
24             } catch (InterruptedException e) {
25             }
26         }
27         this.valor = valor;
28         cheio = true;
29         System.out.println("put " + valor);
30         notifyAll();
31     }
32 }
```

Classe “Produtor.JAVA”

```
1 package ProdutorConsumidor;
2
3 public class Produtor extends Thread {
4     private Buffer buf;
5     private int numero;
6
7     public Produtor(Buffer buf, int numero) {
8         this.buf = buf;
9         this.numero = numero;
10    }
11
12    public void run() {
13        for (int i = 0; i < 5; i++) { // armazena 10 valores no buffer
14            buf.put(i);
15            System.out.println("Produtor #" + numero + " put: " + i);
16            try {
17                sleep((int) (Math.random() * 100));
18            } catch (InterruptedException e) {
19            }
20        }
21    }
22 }
```

Classe: “Consumidor.JAVA”

```
1 package ProdutorConsumidor;
2
3 class Consumidor extends Thread {
4     private Buffer buf;
5     private int numero;
6
7     public Consumidor(Buffer buf, int numero) {
8         this.buf = buf;
9         this.numero = numero;
10    }
11
12    public void run() { // retira 10 valores do buffer
13        int x;
14        for (int i = 0; i < 5; i++) {
15
16            x = buf.get();
17            System.out.println("Consumidor #" + numero + " get: " + x);
18        }
19    }
20 }
```

Classe: “ProdCons.JAVA” contém a *main*

```
1 package ProdutorConsumidor;
2
3 public class ProdCons {
4
5     public static void main(String[] args) {
6         Buffer buf = new Buffer();
7         int i;
8         for (i = 0; i < 1; i++) {
9             new Consumidor(buf, i).start();
10            new Produtor(buf, i).start();
11        }
12    }
13 }
14 }
```

Parte II Thread

- 2) Fazer programas em nas languages Java ou c# ou Python ou C/C++ com as seguintes especificações:
 - Deve-se usar threads para simular a concorrência
 - Usar semáforos para ficar livre de deadlocks.
 - Cada integrante do grupo deve ao menos fazer um vídeo explicando a modelagem do problema e as linhas de execução dos problemas. Assim sendo, o grupo deve disponibilizar um link de acesso aos vídeos.

a) Problema do Barbeiro Dorminhoco

Na barbearia há um barbeiro, uma cadeira de barbeiro e n cadeiras para eventuais clientes esperarem a vez. Quando não há clientes, o barbeiro senta-se na cadeira de barbeiro e cai no sono. Quando chega um cliente, ele precisa acordar o barbeiro. Se outros clientes chegarem enquanto o barbeiro estiver cortando o cabelo de um cliente, eles se sentarão (se houver cadeiras vazias) ou sairão da barbearia (se todas as cadeiras estiverem ocupadas). O problema é programar o barbeiro e os clientes sem cair em condições de disputa (condições de corrida) (*race conditions*). Condições de disputa são situações em que duas ou mais *threads* (ou processos) estão trabalhando juntas e

podem compartilhar algum armazenamento comum que cada uma pode ler e gravar. Uma solução usa três semáforos: *customers*, que conta os clientes à espera de atendimento (exceto o cliente que está na cadeira de barbeiro, que não está esperando); *barbers*, o número de barbeiros (0 ou 1) que estão ociosos à espera de clientes, e *mutex*, que é usado para exclusão mútua. Precisamos ainda de uma variável, *waiting*, que também conta os clientes à espera de atendimento. É essencialmente uma cópia de *customers*. A razão de se ter *waiting* é que não há uma maneira de ler o valor atual do semáforo. Nessa solução, um cliente que entra na barbearia deve contar o número de clientes à espera de atendimento. Se este for menor que o número de cadeiras, ele ficará; do contrário, ele sairá.

Nesta solução, quando chega de manhã para trabalhar, o barbeiro executa o método *barber*, que o leva a bloquear sobre o semáforo *customers*, que inicialmente está em 0. O barbeiro então vai dormir, e permanece dormindo até que o primeiro cliente apareça. Quando chega, o cliente executa *customer* e inicia obtendo *mutex* para entrar em uma região crítica. Se um outro cliente chega logo em seguida, o segundo nada pode fazer até que o primeiro libere o *mutex*. O cliente então verifica se o número de clientes à espera é menor que o número de cadeiras. Se não for, ele liberará o *mutex* e sairá sem cortar o cabelo. Se houver uma cadeira disponível, o cliente incrementará a variável inteira *waiting*. Ele faz então um *up* no semáforo *customers*, que acorda o barbeiro. Nesse ponto, o cliente e o barbeiro estão ambos acordados. Quando o cliente libera *mutex*, o barbeiro o pega, faz alguma limpeza e começa a cortar o cabelo. Quando termina o corte de cabelo, o cliente sai do procedimento e deixa a barbearia. Diferente de nossos exemplos anteriores, não há um laço para o cliente porque para cada um deles terá apenas um corte de cabelo. O barbeiro, contudo, contém um laço para tentar obter o próximo cliente. Se houver algum outro cliente, um novo corte de cabelo será iniciado. Do contrário, o barbeiro cairá no sono.

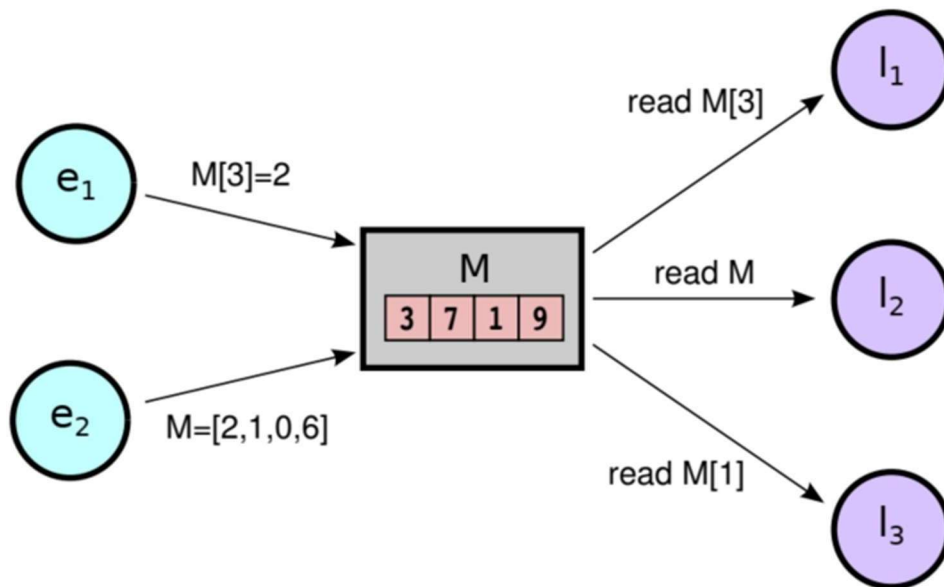


Tanenbaum: Figura 1.20

b) Problema dos Leitores e Escritores

Neste problema, um conjunto de tarefas acessam de forma concorrente uma área de memória compartilhada, na qual podem fazer leituras ou escritas de valores. As leituras podem ser feitas em paralelo, pois não interferem umas com as outras, mas as escritas têm de ser feitas com acesso exclusivo à área compartilhada, para evitar condições de disputa. A Figura a seguir mostra leitores e escritores acessando de forma concorrente um vetor de números inteiros M .

O estilo de sincronização leitores/escritores é encontrado com muita frequência em aplicações com múltiplas threads.



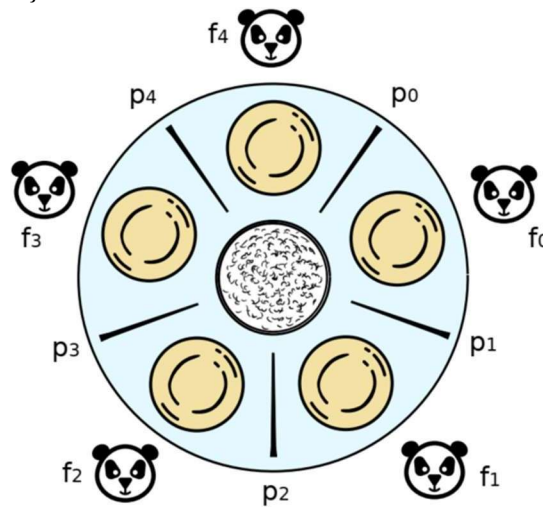
c) Problema dos Fumantes

O problema dos fumantes é um problema clássico de sincronização. Considere a situação com três fumantes (três threads) e um representante de cigarros (um thread). Cada fumante continuamente faz os próprios cigarros e os fuma. Para fazer os cigarros, cada fumante precisa de três ingredientes: tabaco, papel e fósforos. Uma das thread de fumante tem somente papel, a outra tem somente tabaco e a terceira tem somente fósforos. O representante de cigarros tem um estoque infinito dos três materiais. As três threads fumantes estão inicialmente bloqueadas. O representante de cigarros escolhe randomicamente dois ingredientes diferentes, os coloca na mesa e desbloqueia o fumante que tiver o ingrediente restante. O representante então bloqueia. O fumante que está desbloqueado remove os dois ingredientes da mesa, faz o cigarro e o fuma por um tempo aleatório no fumódromo, desbloqueando o representante quando terminar de fumar o cigarro. O representante então coloca outros dois ingredientes na mesa, e o ciclo se repete.

d) Problema dos Filósofos

Um dos problemas clássicos de coordenação mais conhecidos é o jantar dos filósofos, proposto inicialmente por Dijkstra [Raynal, 1986; Ben-Ari, 1990]. Neste problema, um grupo de cinco filósofos chineses alterna suas vidas entre meditar e comer. Há uma mesa redonda com um lugar fixo para cada filósofo, com um prato, cinco palitos (hashis) compartilhados e um grande prato de arroz ao meio. Para comer, um filósofo f_i precisa pegar o palito à sua direita (pi) e à sua

esquerda (p_i+1), um de cada vez. Como os palitos são compartilhados, dois filósofos vizinhos não podem comer ao mesmo tempo. Os filósofos não conversam entre si nem podem observar os estados uns dos outros. A Figura a seguir ilustra essa situação.



e) O jantar dos Selvagens

Uma tribo de selvagens está jantando ao redor de um grande caldeirão contendo N porções de missionário cozido.

Quando um selvagem quer comer, ele se serve de uma porção no caldeirão, a menos que este esteja vazio. Nesse caso, o selvagem primeiro acorda o cozinheiro da tribo e espera que ele encha o caldeirão de volta, para então se servir novamente. Após encher o caldeirão, o cozinheiro volta a dormir.

As restrições de sincronização deste problema são as seguintes:

- Selvagens não podem se servir ao mesmo tempo (mas podem comer ao mesmo tempo);
- Selvagens não podem se servir se o caldeirão estiver vazio;
- O cozinheiro só pode encher o caldeirão quando ele estiver vazio.

f) Problema dos três robôs

Suponha três robôs (Bart, Lisa, Maggie), cada um controlado por sua própria thread. Você deve escrever o código das threads de controle, usando semáforos para garantir que os robôs se movam sempre na sequência Bart → Lisa → Maggie → Lisa → Bart → Lisa → Maggie → (...), um robô de cada vez. Use a chamada `move()` para indicar um movimento do robô. Não esqueça de definir os valores iniciais das variáveis e/ou dos semáforos utilizados. Soluções envolvendo espera ocupada (*busy wait*) não devem ser usadas.